

CSS Grid Layout

What is CSS Grid?

- **CSS Grid** is a 2D layout system (rows + columns).
- Unlike Flexbox (1D), Grid can handle **complex layouts** easily.
- Perfect for web page structures, dashboards, and responsive designs.

Enable Grid

```
.container {  
  display: grid;  
}
```

All direct children of `.container` become **grid items**.

Grid Container Properties

These are applied on the **parent (container)**.

1. **grid-template-columns** → defines column structure.

```
grid-template-columns: 200px 200px 200px; /* 3 fixed columns */  
grid-template-columns: 1fr 2fr;          /* flexible fractions */  
grid-template-columns: repeat(3, 1fr);    /* shorthand */
```

2. **grid-template-rows** → defines row structure.

```
grid-template-rows: 100px 200px;
```

3. **gap** → space between rows/columns.

```
gap: 20px; /* row and column gap */  
row-gap: 10px;  
column-gap: 30px;
```

4. **justify-items** → aligns items horizontally (inside their cells).

```
justify-items: start | end | center | stretch;
```

5. **align-items** → aligns items vertically (inside their cells).

```
align-items: start | end | center | stretch;
```

6. **justify-content & align-content** → align whole grid within container.

```
justify-content: center;  
align-content: space-around;
```

7. **grid-template-areas** → define layout by naming areas.

```
grid-template-areas:
```

```
"header header"
"sidebar main"
"footer footer";
```

Grid Item Properties

These are applied on the **children (items)**.

1. **grid-column & grid-row** → define item position.

```
.item1 {
  grid-column: 1 / 3; /* spans column 1 to 2 */
  grid-row: 1 / 2;
}
```

2. **grid-area** → shorthand for row-start / column-start / row-end / column-end.

```
.item2 {
  grid-area: 2 / 1 / 4 / 3;
}
```

3. **justify-self & align-self** → align single item inside its cell.

```
.item3 {
  justify-self: center;
  align-self: end;
}
```

Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .container {
      display: grid;
      grid-template-columns: 1fr 2fr 1fr;
      grid-template-rows: 80px 200px 80px;
      gap: 10px;
      height: 400px;
    }
    .item {
      background: lightblue;
      border: 1px solid blue;
      text-align: center;
      padding: 20px;
    }
    .header { grid-column: 1 / 4; }
    .sidebar { grid-column: 1 / 2; grid-row: 2 / 3; }
    .main { grid-column: 2 / 4; grid-row: 2 / 3; }
    .footer { grid-column: 1 / 4; }
  </style>
</head>
<body>
  <h2>Grid Example</h2>
  <div class="container">
    <div class="item header">Header</div>
```

```
<div class="item sidebar">Sidebar</div>
<div class="item main">Main Content</div>
<div class="item footer">Footer</div>
</div>
</body>
</html>
```

Key Points

- **Grid** = 2D layout (rows + columns).
- Use **fr** (fraction unit) for flexible layouts.
- Use `grid-template-areas` for named layouts.
- Use **Grid** for page-level layouts, **Flexbox** for content alignment inside.

CSS Media Queries (Responsive Design)

What are Media Queries?

- **Media Queries** allow CSS to adapt styles based on device characteristics like **screen size, orientation, or resolution**.
- They are the foundation of **responsive web design**.

Syntax

```
@media media-type and (condition) {
  /* CSS rules */
}
```

- **media-type** → e.g., screen, print, all.
- **condition** → usually screen width (max-width, min-width).

Common Conditions

1. Max-width

```
@media (max-width: 600px) {
  body {
    background: lightblue;
  }
}
```

Applied only if screen width $\leq$ 600px.

2. Min-width

```
@media (min-width: 768px) {
  body {
    font-size: 18px;
  }
}
```

Applied only if screen width $\geq$ 768px.

3. Range (Min + Max)

```
@media (min-width: 600px) and (max-width: 1200px) {  
  body {  
    background: lightgreen;  
  }  
}
```

4. Orientation

```
@media (orientation: landscape) {  
  body {  
    background: yellow;  
  }  
}
```

5. Print Styles

```
@media print {  
  body {  
    color: black;  
    background: white;  
  }  
}
```

Mobile-First Approach

Start with **small screens** first, then use `min-width` for larger devices.

Example:

```
/* Base styles (mobile first) */  
body {  
  font-size: 14px;  
}  
  
/* Tablet (≥ 768px) */  
@media (min-width: 768px) {  
  body {  
    font-size: 16px;  
  }  
}  
  
/* Desktop (≥ 1024px) */  
@media (min-width: 1024px) {  
  body {  
    font-size: 18px;  
  }  
}
```

Example

```
<!DOCTYPE html>  
<html>  
<head>  
  <style>  
    body {
```

```

        background: lightgray;
        font-family: Arial;
    }
    .box {
        width: 100%;
        padding: 20px;
        text-align: center;
        background: coral;
    }

    /* Tablet */
    @media (min-width: 600px) {
        .box {
            background: lightblue;
            font-size: 20px;
        }
    }

    /* Desktop */
    @media (min-width: 1024px) {
        .box {
            background: lightgreen;
            font-size: 24px;
        }
    }
</style>
</head>
<body>
    <div class="box">Resize the window to see changes</div>
</body>
</html>

```

Key Points

- Use **max-width** → desktop-first design.
- Use **min-width** → mobile-first design (recommended).
- Media queries make layouts responsive across devices.
- Common breakpoints:
 - 576px → Mobile
 - 768px → Tablet
 - 1024px → Small Desktop
 - 1200px+ → Large Desktop

CSS Transitions & Animations

Difference

- **Transition** → Smoothly changes a property **from one value to another** (triggered by an event like hover, focus, or click).
- **Animation** → Allows multiple style changes using **keyframes**, can loop infinitely without user interaction.

CSS Transitions

Syntax

```
selector {  
  transition: property duration timing-function delay;  
}
```

- **property** → which CSS property to animate (e.g., color, width, all).
- **duration** → how long animation takes (e.g., 1s, 500ms).
- **timing-function** → speed curve (linear, ease, ease-in, ease-out, ease-in-out).
- **delay** → wait time before transition starts.

Example

```
<!DOCTYPE html>  
<html>  
<head>  
  <style>  
    .box {  
      width: 100px;  
      height: 100px;  
      background: blue;  
      transition: width 1s ease-in-out, background 0.5s;  
    }  
    .box:hover {  
      width: 200px;  
      background: red;  
    }  
  </style>  
</head>  
<body>  
  <div class="box"></div>  
</body>  
</html>
```

On hover, width expands smoothly and color changes.

CSS Animations

Syntax

```
@keyframes animationName {  
  0%   { property: value; }  
  50%  { property: value; }  
  100% { property: value; }  
}  
  
selector {  
  animation: name duration timing-function delay iteration-count direction  
  fill-mode;  
}
```

Properties

- **name** → keyframe name.
- **duration** → how long 1 cycle runs.
- **timing-function** → speed curve (linear, ease, etc.).
- **delay** → wait before animation starts.
- **iteration-count** → number of repeats (1, infinite).
- **direction** → play direction (normal, reverse, alternate).
- **fill-mode** → final state (none, forwards, backwards, both).

Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .circle {
      width: 100px;
      height: 100px;
      border-radius: 50%;
      background: orange;
      position: relative;
      animation: moveCircle 3s linear infinite alternate;
    }

    @keyframes moveCircle {
      0%   { left: 0;   background: orange; }
      50%  { left: 150px; background: yellow; }
      100% { left: 300px; background: red; }
    }
  </style>
</head>
<body>
  <div class="circle"></div>
</body>
</html>
```

Circle moves left to right, changing color smoothly.

Transition vs Animation Quick Compare

Feature	Transition	Animation
Trigger	Needs event (hover, click)	Can auto-play
Keyframes	Not needed	Required
Complexity	Simple	Complex
Repetition	No loop	Can loop infinitely

Common Uses

- **Transitions** → hover effects, button animations, smooth UI changes.
- **Animations** → banners, loaders, moving objects, attention-grabbing effects.

Transform

- Transform changes an element's **shape, size, rotation, or position**.

Common Functions:

- `translate(x, y)` → move
- `rotate(deg)` → rotate
- `scale(x, y)` → resize
- `skew(x, y)` → tilt
- `matrix()` → combination of all

Example:

```
.box {  
  transform: translateX(50px) rotate(45deg) scale(1.2);  
}
```